

16강

spring MVC MyBatis

이미지 업로드





➤ 게시판에 이미지 업로드 하기

- src/main/java → replyBoard → **WebConfig.java** 클래스 파일을 생성하고 반드시 **WebMvcConfigurer** 인터페이스를 상속(**implements**) 받아 코드를 작성함을 주의하자.
- **WebConfig.java** 코드는 한마디로 "컴퓨터의 실제 폴더(외부 경로)를 웹브라우저에서 접근 가능한 가상 경로로 연결해주는 다리" 역할을 한다.
- 스프링 부트는 기본적으로 보안상 프로젝트 외부의 폴더에 직접 접근하는 것을 차단하지만, 이 설정을 통해 특정 폴더(D:/upload)만 예외적으로 개방하는 것이다.



➤ 게시판 - WebConfig.java 환경 설정파일

```
package replyBoard;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

/**
 * @Configuration: 이 클래스가 스프링 부트의 설정 파일임을 나타낸다.
 * 스프링이 시작될 때 이 클래스를 읽어서 웹 관련 설정을 적용한다.
 * WebMvcConfigurer 인터페이스를 상속받아 스프링 MVC의 기능을 확장한다.
 */
@Configuration
public class WebConfig implements WebMvcConfigurer{

    /**
     * addResourceHandlers: 정적 리소스(이미지, CSS, JS 등)를 관리하는 메서드이다.
     * 외부의 물리적인 경로를 웹에서 사용하는 URL 주소로 매핑하는 설정을 담당한다.
     */

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        // 1. addResourceHandler("/upload/**")
        // - 브라우저 주소창에 "http://localhost:8080/upload/파일명" 이라고 입력했을 때를 의미한다.
        // - "/upload/"로 시작하는 모든 요청(**)을 이 설정이 가로챈다.

        // 2. addResourceLocations("file:///d:/upload/")
        // - 위에서 가로챈 요청을 실제로 어느 폴더에서 찾을지 지정한다.
        // - 반드시 "file:/// " 라는 프로토콜을 붙여야 내 컴퓨터 하드디스크의 실제 경로를 인식한다.
        // - 뒤에 오는 "d:/upload/"는 실제로 파일이 저장되어 있는 물리적인 경로이다.
        registry.addResourceHandler("/upload/**")
            .addResourceLocations("file:///d:/upload/");
    }
}
```



➤ 게시판 – application.properties 수정하기

- 스프링 부트는 기본적으로 파일 업로드 크기 제한이 매우 작게(보통 1MB) 설정되어 있다.
- `src/main/resources/application.properties` 파일에 아래 내용을 추가하여 허용 용량을 늘린다.

파일 하나의 최대 사이즈 (예: 10MB)

```
spring.servlet.multipart.max-file-size=10MB
```

한 번의 요청(파일 여러개 포함)의 전체 최대 사이즈 (예: 20MB)

```
spring.servlet.multipart.max-request-size=20MB
```



➤ 게시판 – replyBoard DB 테이블 수정하기

- 아래, 이미지 파일을 저장할 upload1, upload2 필드명을 추가 작성한다.

```
create table replyBoard(  
  num int auto_increment primary key,  
  writer varchar(20),  
  email varchar(50),  
  subject varchar(50),  
  password varchar(10),  
  reg_date datetime default now(),  
  ref int,  
  re_step int,  
  re_level int,  
  readcount int default 0,  
  content varchar(1000),  
  upload1 varchar(50),  
  upload2 varchar(50)  
);
```



➤ 게시판 – replyBoardDTO 수정하기

- src/main/java → replyBoard → dto → ReplyBoardDTO 클래스에 upload1, upload2 필드명을 추가 작성한 후, 반드시 getter, setter을 진행한다.

```
public class ReplyBoardDTO {  
    private int num; // 번호  
    private String writer; // 작성자  
    private String email; // email  
    private String subject; // 제목  
    private String password; // 비밀번호  
    private Date reg_date; // 작성일  
    private int ref; // 같은 원글 번호 (댓글 묶음 기준)  
    private int re_step; // 댓글 출력 순서 (위 → 아래)  
    private int re_level; // 댓글 들여쓰기 단계 (원글/댓글 구분)  
    private int readcount; //조회수  
    private String content; // 글 내용  
    private String upload1; // 이미지1  
    private String upload2; // 이미지2  
}
```



➤ 게시판 – replyboardWrite_form 수정하기

- src/main/resources → templates → replyBoard → replyboardWrite_Form.html 파일의 `<form></form>` 태그안의 이미지를 전송하는 코드인 `enctype="multipart/form-data"`를 추가한다.

```
<div class="container">
  <h2> 게시글 쓰기 </h2>
  <form th:action="@{/board/writerPro}" method="POST" name="formname" enctype="multipart/form-data">
    <table width="600" border="1">
      <tr height="40">
        <td align="center" width="150">작성자</td>
        <td width="450"><input type="text" name="writer"></td>
      </tr>
    </table>
  </form>
</div>
```

`enctype="multipart/form-data"`는 "이 양식(form)에는 글자뿐만 아니라 파일(이미지, 문서 등)도 섞여서 전송된다"는 것을 서버에 알려주는 설정이다.



➤ 게시판 – replyboardWrite_form 수정하기

- src/main/resources → templates → replyBoard → replyboardWrite_Form.html 파일의 `<form></form>` 태그안의 이미지를 입력하는 `<input>` 태그를 추가한다.

```
<tr height="40">
  <td align="center" width="150">글내용</td>
  <td width="450"><textarea rows="10" cols="55" name="content" class="b_contents"></textarea>
</tr>
<!-- 이미지 업로드 -->
<tr height="40">
  <td align="center" width="150">이미지1</td>
  <td width="450">
    <input type="file" name="file1">
  </td>
</tr>

<tr height="40">
  <td align="center" width="150">이미지2</td>
  <td width="450">
    <input type="file" name="file2">
  </td>
</tr>
<!-- 이미지 업로드 -->
```



➤ 게시판 – replyboardList 폼 수정하기

- src/main/resources → templates → replyBoard → replyboardList.html 파일에 입력한 이미지를 썸네일로 출력하는 태그를 추가한다.

```
<tr th:each="li, stat:${rlist}">
<td th:text="${stat.size - stat.index}"></td>
<td style="text-align:left;">

    <!-- 썸네일 이미지 -->
    

    <!-- 답글인 경우 -->
    <span th:if="${li.re_level > 1}"
        th:style="'padding-left:' + (${li.re_level - 1} * 20) + 'px'">
        ↳[re]
    </span>

    <a th:href="@{/board/detail(num=${li.num})}"
        th:text="${li.subject}"></a>
</td>
```



➤ 게시판 – replyboard-mapper.xml 수정하기

- src/main/resources → mybatis → mappers → replyboard-mapper.xml 파일에 insert SQL문에 입력한 이미지파일이 추가되도록 upload1, upload2 필드명을 추가한다.

```
<!-- 테이블 추가 -->
<!--
    ref=0; //글그룹 = 쿼리 실행시 가장 큰 ref값을 가져온 후 더하기 1을 한다.
    re_step=1; //새로운 글은 부모글이므로 1
    re_level=1;
-->
<insert id="insertReplyBoard" parameterType="rDTO">
    INSERT INTO replyBoard(writer,email,subject,password,ref,re_step,re_level,content,upload1,upload2)
    VALUES(#{writer},#{email},#{subject},#{password},
    <!-- 별칭 'r'를 주어 별도 쿼리로 인식시킴 -->
    (SELECT IFNULL(MAX(r.ref) + 1, 1) FROM replyBoard r),1,1,#{content},#{upload1},#{upload2})
</insert>
```



➤ 게시판 – replyBoardController 수정하기

- src/main/java → replyBoard → controller → ReplyBoardController.java 클래스에 글 쓰기를 처리하는 컨트롤러인 /board/WritePro를 아래와 같이 수정하여 이미지를 업로드하도록 작성한다.
- 파일 업로드는 반드시 @PostMapping() 어노테이션을 이용해 작성한다.
- **@PostMapping**: HTML의 <form entype="multipart/form-data">를 통해 들어온 데이터를 처리한다.
- **@RequestParam("file1") MultipartFile**: <input type="file" name="file1">에 담긴 파일을 받는 파라미터이다.



➤ 게시판 – replyBoardController 수정하기

```
// 파일업로드는 PostMapping()이다.
@PostMapping("/board/writerPro")
public String boardWriterPro(Model model, ReplyBoardDTO rdto,
    @RequestParam("file1") MultipartFile upload1,
    @RequestParam("file2") MultipartFile upload2
    ) throws IllegalStateException, IOException {

    System.out.println("ReplyBoardController boardWriterPro() 호출 함");

    // 1. 파일을 저장할 실제 하드디스크의 위치를 지정한다.
    // WebConfig에서 설정한 'file:///d:/upload/' 경로와 반드시 일치해야 웹에서 불러올 수 있다.
    String savePath = "d:/upload/";

    // 2. [안전장치] 만약 'd:/upload/' 폴더가 없으면 프로그램을 통해 자동으로 생성한다.
    File saveDir = new File(savePath);
    if (!saveDir.exists()) {
        saveDir.mkdirs(); // mkdirs()는 상위 폴더가 없어도 한꺼번에 다 만들어준다
    }
}
```



➤ 게시판 – replyBoardController 수정하기

```
// ===== 3. 첫 번째 이미지(upload1) 업로드 처리 =====
if (!upload1.isEmpty()) { // 사용자가 파일을 실제로 선택해서 보냈는지 확인한다.

    // 사용자가 올린 원래 파일명 (예: "my_car.jpg")을 가져온다.
    String originalName1 = upload1.getOriginalFilename();

    // [중복 방지] 파일명이 겹치지 않게 UUID를 생성한다.
    // substring(0, 4)를 사용해 36자리 중 앞 4자리만 가져와서 파일명을 짧게 만든다. (예: "a1b2_my_car.jpg")
    String saveName1 = UUID.randomUUID().toString().substring(0, 4) + "_" + originalName1;

    // savePath(경로)와 saveName1(이름)을 합쳐서 실제 저장될 파일 객체를 생성한다.
    File file1 = new File(savePath + saveName1);
    // File file1 = new File("c:/upload/" + saveName1);

    // transferTo(): 이 명령어가 실행되는 순간 서버의 메모리에 있던 파일이
    //                 실제 하드디스크(D:/upload/)로 복사된다.
    upload1.transferTo(file1);

    // 📁 DB에 저장할 파일명 DTO에 세팅
    // [핵심] 하드디스크에 저장된 '새 파일명'을 DTO 객체에 담는다.
    // 그래야 나중에 DB의 upload1 컬럼에 이 이름이 기록된다.
    rdto.setUpload1(saveName1);
}
```



➤ 게시판 – replyBoardController 수정하기

```
// ===== 4. 두 번째 이미지(upload2) 업로드 처리 (방식은 위와 동일) =====  
if (!upload2.isEmpty()) {  
    String originalName2 = upload2.getOriginalFilename();  
    String saveName2 = UUID.randomUUID().toString().substring(0, 4) + "_" + originalName2;  
  
    File file2 = new File(savePath + saveName2);  
    upload2.transferTo(file2);  
  
    rdto.setUpload2(saveName2);  
}  
  
// 5. 게시글 내용(제목, 작성자 등)과 함께 위에서 세팅한 '파일명'들을 DB에 최종 저장합니다.  
replyBoardService.insertReplyBoard(rdto);  
// 6. 모든 작업이 끝나면 게시글 목록 페이지로 화면을 이동시킵니다.  
return "redirect:/board/list";  
}
```

➤ **게시판 출력 화면**



➤ replyBoard DB 테이블

[illegible]



➤ 게시판에 이미지 업로드 하기 총 정리

단계	주요 코드 (명령어)	설명 및 학생들을 위한 핵심 요약
1. 데이터 수신	@RequestParam("file1") MultipartFile	HTML의 <input type="file">에 담긴 실제 파일 데이터 를 서버로 가져오는 바구니 역할을 합니다.
2. 경로 설정	String savePath = "d:/upload/";	파일이 내 컴퓨터 하드디스크 어디에 저장될지 물리적인 주소 를 정합니다.
3. 폴더 생성	saveDir.mkdirs();	설정한 경로에 폴더가 없을 경우, 에러 방지를 위해 폴더를 자동으로 생성 해주는 안전장치입니다.
4. 중복 방지	UUID.randomUUID().toString()	똑같은 이름의 파일이 올라오면 덮어쓰기될 위험이 있어, 세상에 하나뿐인 고유한 이름 을 만들어줍니다.
5. 이름 가공	.substring(0, 4)	너무 긴 UUID(36자리)를 앞에서 4글자만 잘라내어 DB 관리와 가독성을 좋게 만듭니다.
6. 파일 저장	upload1.transferTo(file1);	가장 중요한 명령어! 메모리에 떠 있던 파일을 실제 하드디스크 폴더로 완전히 복사 하여 저장합니다.
7. DB 준비	rdto.setUpload1(saveName1);	하드디스크에 저장된 '새로운 이름'만 DTO에 기록 합니다. (실제 파일을 DB에 넣는 게 아님!)
8. 최종 저장	insertReplyBoard(rdto);	파일명과 게시글 텍스트 정보를 묶어 데이터베이스(DB)에 최종적으로 등록 합니다.